UnionLabs SDR Platform — File Index & Command Mapping

Contents

File Manifest

A one-line description of every meaningful file. The per-scheme `configs/*.json` and the runtime `phy_outputs/**` dumps are collapsed (they are repetitive by design).

Root — instruction files & entry scripts

File	Description
`README.md`	Project overview: quick start, layout, three ways to use the PHY, and the router to the commar
`HOW_TO_ADD_ALGORITHM.md`	Step-by-step guide to authoring/uploading your own algorithm (`app.py` + `make()`).
`STRUCTURE.md`	The full repo layout and how the auto-generated pieces are produced.
`PARAMETERS.md`	Auto-generated reference of every controllable `sdr_system` option (JSON/CLI/Python).
`SYSTEM_REFERENCE.md` / `.pdf`	The deep engine reference — the math and every algorithm in the PHY.
`COMMANDS.md`	Ready-to-run, per-scheme TX/RX command pairs with tuned gains.
`COMMANDS_FEC_TESTS.txt`	Command sheet for the LDPC/turbo FEC hardware tests.
`USRP_CARRIER_MODULATION.txt`	Per-device (B210/N210/X310) command reference across carriers/modulations.
`HARDWARE.md`	Hardware setup, radio inventory, and per-device parameters.
`MANIFEST.md`	This file — a one-line description of every file.
`run.sh`	One command to run an algorithm over the PHY (defaults + role/channel passthrough).
`radio.sh`	One command to run raw TX/RX on a USRP (B210 default; `--device n210|x310`).

Command interfaces — the CLI and the Python (`sdr.py`) version

The command files above (`COMMANDS.md`, `COMMANDS_FEC_TESTS.txt`, `USRP_CARRIER_MODULATION.txt`) list raw **`sdr_system` CLI** commands. The equivalent **Python (`sdr.py`)** commands — the prior interface users worked with — are shown here alongside. **Mapping rule:** every CLI `--option value` becomes the Python keyword `option=value` (hyphens `->` underscores); a bare `--flag` becomes `flag=True`; the `--role` picks the helper (`tx`/`rx`/`both`/`sink_arq`/`source_arq`). Run the Python forms from `drivers/usrp/python/` (`from sdr import ...`).

Transmit only (B210)

```
./sdr_system --role tx --tx-args serial=30CD424 --scheme QPSK --tx-gain 78 --fec true
```

```
tx(tx_args="serial=30CD424", scheme="QPSK", tx_gain=78, fec=True).run()
```

Receive only (B210)

```
./sdr_system --role rx --rx-args serial=30CD3F7 --scheme QPSK --rx-gain 20 --fec true
```

```
rx(rx_args="serial=30CD3F7", scheme="QPSK", rx_gain=20, fec=True).run()
```

Stop-and-wait ARQ pair (RX = sink, TX = source; start the sink first)

```
./sdr_system --role sink_arq   --rx-args serial=30CD3F7 --scheme QPSK --fec true
./sdr_system --role source_arq --tx-args serial=30CD424 --scheme QPSK --fec true
```

```
run_pair(sink_arq(rx_args="serial=30CD3F7", scheme="QPSK", fec=True),
         source_arq(tx_args="serial=30CD424", scheme="QPSK", fec=True))
# or from a config:  python3 run.py configs/qpsk_arq_pair.json
```

N210 example (exact rates + DQPSK, from `USRP_CARRIER_MODULATION.txt`)

```
./sdr_system --role rx --rx-args addr=192.168.20.2 --rx-subdev A:0 --rx-freq 915e6 \
             --rx-rate 2e6 --tx-rate 2e6 --symbol_rate 1e6 --rx-gain 25 --scheme DQPSK
```

```
rx(rx_args="addr=192.168.20.2", rx_subdev="A:0", rx_freq=915e6,
   rx_rate=2e6, tx_rate=2e6, symbol_rate=1e6, rx_gain=25, scheme="DQPSK").run()
```

Channel sense / print-only / background

```
sense_channel(rx_args="serial=30CD3F7")          # channel_sense.py — energy sensing
tx(scheme="QPSK").command()                       # print the CLI string, don't run
source_arq(tx_args="serial=30CD424").popen()      # launch in the background
```

`sdr.py` exposes **every** option this way (it is auto-generated from `sdr_system --help`); the full option list is in `PARAMETERS.md`. `radio.sh` and `run.sh` wrap these for one-line use.

`experiments/` — user algorithms (uniform API)

File	Description
`README.md`	The upload folder + the worked-examples table; points to `HOW_TO_ADD_ALGORITHM.md` for the how-to
`_template/app.py`	Copy-me skeleton showing the minimal `transmit/receive` + `make()` form.
`plain_echo/echo_algo.py`	A pure, framework-agnostic echo algorithm (no PHY imports).
`plain_echo/app.py`	The 2-line `make()` binding that links `echo_algo.py` to the framework.
`echo/app.py`	Round-trip smoke test written as an `SdrApp` subclass (the advanced form).
`fl/app.py`	Federated learning as an algorithm (transmit model vector / receive aggregate).
`clip_semcom/app.py`	CLIP semantic comm as an algorithm (transmit embedding / receive-and-classify).
`marl_multi/app.py`	**Real multi-agent** random access — N independent A2C agents contend for one AP (uses the `run_s`
`marl/app.py`	Single-agent random access with **online A2C learning** — reuses the real `Actor/Critic`; learns tran

`phy/` — the PHY layer (top level)

File	Description
`GUIDE.md`	Beginner's guide to the PHY-layer codes and every way to run them.
`CMakeLists.txt`	Build for `sdr_system`; POST_BUILD regenerates `sdr.py` + `PARAMETERS.md`.
`phy.cfg`	Fully-defaulted config-file template (every option = its default).

`drivers/usrp/python/` — the Python you call

File	Description
`run_algo.py`	Loads an uploaded algorithm and runs it over the PHY (uniform-API runner).
`phy_link.py`	The uniform framework: contract, `adapt()`, `Codec`, channels, round-trip drivers.
`sdr.py`	Auto-generated wrapper exposing every `sdr_system` option as a Python kwarg.
`run.py`	Runs the PHY from a JSON config (single or paired TX/RX).
`configs/*.json`	Ready-made run configs (per-scheme ARQ pairs, tone TX, tone monitor).
`example.py`	Hand-written usage examples that print commands without hardware.
`phy_flow_example.py`	Worked `pyphy` block flowgraph (SC + OFDM, end-to-end).
`channel_sense.py`	Energy-based channel-occupancy sensing (sense->decide loop).
`freq_scan.py`	Sweep a band and report power per frequency to pick a clean carrier.
`power_monitor.py`	Monitor received power over time.
`ber_monitor.py`	Long-term timestamped per-burst BER/detection trajectory (CSV + plot).
`ber_dist.py`	Compare BER distributions across monitor runs (histograms/CDF).
`range_ber.py`	Two-host range-test BER logger.
`tx_repeats_viz.py`	Visualize repeated-transmission behavior.
`phy_outputs/**`	Runtime `--viz` signal dumps per scheme (tx/rx waves + symbols; gitignored).

`drivers/usrp/bindings/` — the block API

File	Description
`pyphy.cpp`	pybind11 module exposing DSP stages (modulate/FEC/sync/OFDM/Radio) as numpy functions.
`build.sh`	Builds the `pyphy` extension (add `WITH_UHD=1` for the Radio block).

`drivers/usrp/src/` — C++ engine

File	Description
`main.cpp`	CLI parsing + role orchestration (the `sdr_system` entry point).
`modulator.cpp`	Modulation/demodulation implementation.
`filters.cpp`	RRC pulse-shaping / matched / polyphase filter implementation.
`synchronization.cpp`	Acquisition/preamble synchronization implementation.
`transceiver.cpp`	UHD transmit/receive streaming implementation.

`drivers/usrp/include/` — C++ headers

File	Description
`physical_layer.hpp`	Top-level PHY orchestration (config,
`transceiver.hpp`	UHD device wrapper (open/stream/
`modulator.hpp` / `modulator_extended.hpp`	Core constellations (PSK/QAM/diffe
`filters.hpp` / `taps.hpp`	Polyphase/RRC filters / precomputed
`synchronization.hpp` / `frequency_offset.hpp` / `phase_offset.hpp` / `timing_recovery.hpp`	ACQ sync / CFO / carrier-phase PLL
`channel_estimation.hpp`	Pilot-based channel estimation.
`fec.hpp` / `ldpc.hpp` / `turbo.hpp`	FEC selector (conv/ldpc/turbo) / LD
`ofdm.hpp` / `ofdm_pipeline.hpp` / `fft.hpp`	OFDM mod/demod / threaded OFDI
`messages.hpp` / `ACQ_stop_and_wait.hpp` / `ack_transport.hpp`	Framing+CRC / stop-and-wait ARQ
`net.hpp` / `FIFO.hpp`	TCP socket helpers / thread-safe FIF
`lora.hpp`	LoRa/CSS chirp waveform (transmit

File	Description
viz.hpp	Capture TX/RX signals to phy_output

drivers/usrp/tools/ — codegen, docs, plotting

File	Description
gen_python_api.py	Parses sdr_system --help -> ge
gen_config_template.py	Generates the fully-defaulted p
md_glyph_fix.py	Preprocesses Markdown so all
build_reference_pdf.sh	Renders SYSTEM_REFERENCE.md -
build_applications_pdf.sh	Renders APPLICATIONS_INTRO.mc
make_reference_figures.py	Generates the explanatory plot
plot_viz.py / plot_evidence*.py / plot_fec_*.py / plot_waveform_snr.py / ofdm_spectrum.py	Plotting utilities for captured si
lora_loopback_test.cpp	Radio-free LoRa waveform loo

drivers/usrp/sim/ and drivers/usrp/tests/

File	Description
sim/tx_app.cpp · sim/rx_app.cpp · sim/build.sh · sim/README.md	Two-terminal TX/RX DSP demo over a localhost socket (no r
tests/*.cpp	Radio-free unit/integration tests, one per subsystem (fec/ldp
tests/stub/transceiver.hpp	No-radio stub of the transceiver so tests build without UHD.
tests/run_demo.sh	Runs a test demo end to end.

experiments/ — worked apps

File	Description
APPLICATIONS_INTRO.md / .pdf	Introduces each applicat
EXPERIMENT_GUIDE.md / .pdf	Step-by-step operating r
CROSS_TESTBED_FL.md	Design rationale for cros
marl_ra/marl_ra.py … MARL_base/network/learning/setting_Union.py	The RL code: environme
marl_ra/marl_phy.py · real_channel.py	PHY bridge (warm sourc
marl_ra/ap_multi.py · agent_node.py · slot_sync.py · mock_medium.py	Multi-agent AP + ACK r
marl_ra/marl_env.py · marl_train.py · marl_multi_env.py · marl_multi_train.py · aloha_baseline.py	Single-agent env+traine
marl_ra/README.md · INTEGRATION.md · requirements.txt · run_training.sh	Overview / design note
fl/fl.py · fl_core.py · mnist_sgd_over_sdr.py	FedAvg app / shared FL
clip_semcom/semcom.py · semcom_core.py · phy_port.py	CLI app / CLIP backend
clip_semcom/README.md · INTEGRATION.md · requirements.txt	Overview / integration c
stc_aircomp/README.md · INTEGRATION.md	Design + build plan for
jammer/jammer.py · README.md	Configurable RF interfer

deploy/ — Docker & install

File	Description
initialization.sh	Installs the toolchain (--build also compiles drivers/usrp/bu
run_sink.sh · run_source.sh · fec_test.env	One-line FEC-test wrappers for the sink/source + their share
Dockerfile · Dockerfile.novnc · DOCKER.md	Container image for the PHY / VNC variant / container docs

File	Description
`docker/*.sh` · `launch.py` · `reservation.example.json` · `README.md`	Container build/run scripts, a launcher, an example reservat

`docs/` — assets (non-instruction)

File	Description
`CHANGES.md`	Project changelog / history.
`make_stack_variants.py` · `make_framework_diagram.py`	Generators for the architecture diagrams.
`slides/make_slides.py`	Generator for the overview slide deck.
`sdr_stack.*` · `fig1_variant_*` · `framework_marl.*` · `SDR_Stack_slides.*`	Rendered architecture figures and slide decks.